Code Source Python

Repertoire: C:\Users\Jean-Baptiste\Desktop\Travail\CReAAH\PyknFit\PyknFit

Date: 2026-09-22 15:04:34

Fichiers inclus: 20

Fichiers exclus: 1

Liste des fichiers inclus:

1. app.py
2. data.py
3. i18n.py
4. install.py
5. main.py
6. plotting.py
7. test_engine.py
8. ui.py
9. pyknfit__init__.py
10. pyknfit\diagnostics\metrics.py
11. pyknfit\diagnostics__init__.py
12. pyknfit\distributions\base.py
13. pyknfit\distributions\gaussian.py
14. pyknfit\distributions\lognormal.py
15. pyknfit\distributions\weibull.py
16. pyknfit\distributions__init__.py
17. pyknfit\mixtures\mixture_model.py
18. pyknfit\mixtures__init__.py
19. pyknfit\utils\conversions.py
20. pyknfit\utils__init__.py

Fichier: app.py

Chemin: .\app.py

```
import os
import tkinter as tk
from ui import UIManager
from data import DataManager
from plotting import PlotManager
from pyknfit.distributions import DISTRIBUTIONS
from pyknfit.mixtures.mixture_model import MixtureModel
from pyknfit.distributions.gaussian import Gaussian
from i18n import LANGUAGES

class PyknFitApp:
    def __init__(self, root):
        self.lang = "fr"
        self.use_lognormal = True
        self.current_distribution_type = 'Gaussienne'

        self.root = root
        self.root.title(self.t("app_title"))
        current_dir = os.path.dirname(os.path.abspath(__file__))
        self.logo_path = os.path.join(current_dir, "logo.png")
        self.root.iconphoto(False, tk.PhotoImage(file=self.logo_path))

        self.ui = UIManager(self)
        self.data = DataManager(self)
        self.plot = PlotManager(self)
        self.model = MixtureModel(self, [])

        self.init_attributes()          # <-- AVANT les créations UI

        self.ui.create_widgets()
        self.ui.create_menu()
        self.ui.create_context_menu()

    def t(self, key):
        return LANGUAGES[self.lang].get(key, key)

    def set_language(self, lang):
        self.lang = lang
        self.refresh_ui()

    def refresh_ui(self):
        self.root.title(self.t("app_title"))

        # Recréer la BooleanVar en lisant l'état persistant
        self.var_include_model_info = tk.BooleanVar(value=self.m_include_model_info)

        for widget in self.root.winfo_children():
            widget.destroy()

        self.ui.create_widgets()
        self.ui.create_menu()
        self.ui.create_context_menu()

    def init_attributes(self):
        self.m_chiffresX_μ = []
        self.m_chiffresY_μ = []
        self.m_chiffresX_phi = []
        self.m_chiffresY_phi = []
        self.m_init_centers_selected = []
        self.m_centres_microns = []
```

```
            self.m_centres_phi = []
            self.m_proportions = []
            self.m_sigmas = []
            self.m_min_sigma = 0.1
            self.m_min_area_ratio = 0.01
            self.m_amplitudes = []
            self.m_last_error_metrics = None
            self.m_tous_les_resultats = []
            self.m_model = None
            self.m_model_params = []
            self.m_components_model = None
            self.m_sampling_model_size = 1000
            self.m_titre_graphique = ""
            self.m_fig1_X = None
            self.m_fig1_Y = None
            self.m_best_fit_curve = None
            self.m_ticks = [-1.5849, 0, 3.3219, 6.6439, 9.9658, 13.2877, 16.6096]
            self.m_ticklabels = ['3000', '1000', '100', '10', '1', '0.1', '0.01']

            self.m_ax0 = None
            self.m_ax2 = None
            self.m_fig0 = None
            self.m_fig2 = None

            self.m_dynamic_x = []
            self.m_dynamic_y = []
            self.m_edit_point_menu_open = False
            self.m_annotations = {}

            self.bouton_calculer = None
            self.distribution_type = None
            self.bouton_calculer_visible = False
            self.m_include_model_info = True  # coché par défaut = comportement actuel

            if hasattr(self.ui, "graph_frame"):
                for widget in self.ui.graph_frame.winfo_children():
                    widget.destroy()

        def create_distribution(self, dist_type):
            cls = DISTRIBUTIONS.get(
                dist_type,
                DISTRIBUTIONS["Gaussienne"]
            )
            return cls()
```

Fichier: data.py

Chemin: .\data.py

```
import os
import numpy as np
from datetime import datetime
import pandas as pd
import matplotlib.pyplot as plt
import tkinter.simpledialog as sd
from tkinter import filedialog, messagebox, Tk
import tkinter as tk
from tkinter import ttk

class DataManager:
    def __init__(self, app):
        self.app = app

    def load_data(self):
        root = Tk()
        root.withdraw()
        fichier_path = filedialog.askopenfilename(
            title=self.app.t("choose_file"),
            filetypes=[("Fichiers Excel", "*.xlsx *.xls"), ("Fichiers CSV", "*.csv"), ("Tous le...
        )
        if not fichier_path:
            print(self.app.t("no_file_selected"))
            return None

        try:
            if fichier_path.endswith(('.xlsx', '.xls')):
                self.app.m_imported_data = pd.read_excel(fichier_path)
            elif fichier_path.endswith('.csv'):
                try:
                    self.app.m_imported_data = pd.read_csv(fichier_path, sep="\t")
                except:
                    self.app.m_imported_data = pd.read_csv(fichier_path, sep=";")
            else:
                print("Format non supporté.")
                return None

            self.app.m_imported_data = self.app.m_imported_data.map(
                lambda x: float(str(x).replace(",", ".")) if isinstance(x, str) and x.replace(",...
            )

            self.app.selection_echantillon.config(state="normal")
            self.app.selection_echantillon["values"] = list(self.app.m_imported_data.columns[1:])
            self.app.selection_echantillon.current(0)
            self.app.selection_echantillon.bind("<<ComboboxSelected>>", self.update_input_data_...

            self.update_input_data(0)

        except Exception as e:
            print(f"Erreur import : {e}")
            return None

    def update_input_data(self, idx):
        l_x_data = '\n'.join([str(round(x, 10)).replace('.', ',') for x in self.app.m_imp...
        l_y_data = '\n'.join([str(round(x, 10)).replace('.', ',') for x in self.app.m_imported_...
        l_titre = self.app.m_imported_data.columns[idx + 1]
        self.insert_input_data(l_titre, l_x_data, l_y_data)

    def update_input_data_on_change(self, event):
        if hasattr(self.app.ui, "graph_frame"):
```

```
                for widget in self.app.ui.graph_frame.winfo_children():
                    widget.destroy()

            self.app.m_dynamic_x = []
            self.app.m_dynamic_y = []
            self.app.m_init_centers_selected = []
            self.app.m_annotations = {}
            self.app.m_model = None
            self.app.m_components_model = None
            self.app.m_last_error_metrics = None

            self.update_input_data(self.app.selection_echantillon.current())

        def insert_input_data(self, _title="Test", _x_data="0,03\n0,04\n0,07\n0,1\n0,2\n0,3\n...
            self.app.m_titre_entry.delete("0", "end")
            self.app.m_titre_entry.insert(0, _title)
            self.app.m_textX.delete("1.0", "end")
            self.app.m_textX.insert("1.0", _x_data)
            self.app.m_textY.delete("1.0", "end")
            self.app.m_textY.insert("1.0", _y_data)

        def enregistrer_graphique(self):
            if self.app.m_fig2:
                l_titre = self.app.m_titre_graphique + "_" + datetime.now().strftime("%d-%m-%Y")
                fichier = filedialog.asksaveasfilename(
                    defaultextension=".png",
                    filetypes=[("PNG", "*.png"), ("JPEG", "*.jpg"), ("PDF", "*.pdf"), ("SVG", ...
                    initialfile=l_titre
                )
                if fichier:
                    ext = os.path.splitext(fichier)[1].lower()
                    if ext in [".png", ".jpg", ".svg"]:
                        self.app.m_fig2.savefig(fichier, dpi=300, bbox_inches='tight')
                        messagebox.showinfo("Enregistrement", self.app.t("graph_saved")+f"'{fichier}'")
                    elif ext == ".pdf":
                        from matplotlib.backends.backend_pdf import PdfPages
                        with PdfPages(fichier) as pdf:
                            pdf.savefig(self.app.m_fig2, bbox_inches='tight', dpi=300)
                            d = pdf.infodict()
                            d['Title'] = f'PyknFit - {self.app.m_titre_graphique}'
                            d['Author'] = 'PyknFit'
                            d['Subject'] = 'Décomposition multimodale'
                            d['CreationDate'] = datetime.now()
                    else:
                        messagebox.showerror("Erreur", self.app.t("unsupported_format")+f" : {ext}")
            else:
                messagebox.showinfo("Info", self.app.t("error_graph"))

        def enregistrer_resultats_decomposition(self):
            if not self.app.m_tous_les_resultats:
                messagebox.showinfo("Info", self.app.t("no_results"))
                return

            df = pd.DataFrame(self.app.m_tous_les_resultats)
            l_titre = 'Résultats_décomposition_' + datetime.now().strftime("%d-%m-%Y")
            fichier = filedialog.asksaveasfilename(
                defaultextension=".xlsx",
                filetypes=[("Excel", "*.xlsx")],
                initialfile=l_titre
            )
            if fichier:
                df.to_excel(fichier, index=False)
                messagebox.showinfo("Enregistrement", self.app.t("results_saved")+ f" '{fichier}'")

        def enregistrer_points_gaussiennes_excel(self):
```

```
            if self.app.m_model is None or self.app.m_fig1_X is None:
                messagebox.showinfo("Info", self.app.t("no_model"))
                return

            fichier = filedialog.asksaveasfilename(
                defaultextension=".xlsx",
                filetypes=[("Excel", "*.xlsx")],
                initialfile=f"Points_gaussiennes_{self.app.m_titre_graphique}"
            )
            if not fichier:
                return

            import tkinter.simpledialog as sd

            sample_size = sd.askinteger(
                self.app.t("sampling"),
                self.app.t("num_points"),
                initialvalue=self.app.m_sampling_model_size,
                minvalue=50,
                maxvalue=50000
            )
            if sample_size is None:
                return

            self.app.m_sampling_model_size = sample_size

            x_dense = np.linspace(
                min(self.app.m_fig1_X),
                max(self.app.m_fig1_X),
                self.app.m_sampling_model_size
            )

            components_eval = self.app.m_model.eval_components(x=x_dense)
            best_fit = self.app.m_model.eval(x=x_dense)

            x_dense = x_dense[::-1]
            best_fit = best_fit[::-1]
            for rank, (name, y_component) in enumerate(components_eval.items(), start=1):
                components_eval[name] = y_component[::-1]

            X_microns = 1000 * (2 ** (-x_dense))

            data = {
                "X_phi": x_dense,
                "X_µm": X_microns,
                "Best_fit": best_fit
            }

            for rank, (name, y_component) in enumerate(components_eval.items(), start=1):
                data[f"G{rank}"] = y_component

            df = pd.DataFrame(data)
            df.to_excel(fichier, index=False)

            messagebox.showinfo(
                self.app.t("save_success"),
                self.app.t("save_points") + f"\n{fichier}"
            )
```

Fichier: i18n.py

Chemin: .\i18n.py

```
    LANGUAGES = {
        "fr": {
            # === Application ===
            "app_title": "Pyk?n?Fit",

            # === UI principale ===
            "select_sample": "Sélectionner échantillon :",
            "sample_title": "Titre de l'échantillon :",
            "show_data": "Afficher les données",
            "calculate_model": "Calculer le modèle",

            # === Menus ===
            "file": "Fichier",
            "process": "Traitement",
            "language": "Langue",
            "quit": "Quitter",

            "import_data": "Importer des données",
            "generate_example": "Générer des données exemples",

            "save_full_graph": "Enregistrer le graphique complet",
            "save_decomposition_results": "Enregistrer les résultats des décompositions",
            "save_gaussian_points": "Enregistrer les points des gaussiennes (Excel)",

            # === Context menu ===
            "edit": "Éditer",
            "delete": "Supprimer",
            "clear": "Effacer",
            "cut": "Couper",
            "copy": "Copier",
            "paste": "Coller",

            # === Graphes ===
            "left_click_help": "clic gauche = ajout",
            "right_click_help": "clic droit = éditer / supprimer",
            "x_label_phi": "x (?)",
            "y_label_frequency": "Fréquence",
            "particle_diameter": "Diamètre des particules en µm (log)",
            "data": "Données",
            "best_fit": "Ajustement",
            "absolute_error": "Erreur absolue",
            "relative_error": "Erreur relative",
            "MSE": "MSE",
            "R_square": "R²",

            # === Modélisation ===
            "multi_modal_title": "Décomposition multimodale de l'échantillon",

            # === Boîtes de dialogue ===
            "error": "Erreur",
            "info": "Information",
            "warning": "Attention",

            "error_graph": "Une erreur est survenue lors de la génération du graphique :",
            "invalid_data": "Les données saisies sont invalides.",
            "need_data": "Vous devez avoir saisi des données.",
            "need_one_center": "Vous devez avoir au moins un centre.",
            "model_success": "Décomposition probabiliste terminée",

            "no_file_selected": "Aucun fichier sélectionné.",
```

```
            "no_results": "Aucun résultat stocké pour le moment.",
            "no_model": "Aucun modèle calculé.",
            "graph_saved": "Graphique sauvegardé sous",
            "results_saved": "Résultats sauvegardés sous",

            # === Import / Export ===
            "choose_file": "Choisissez un fichier Excel ou CSV",
            "unsupported_format": "Format non supporté",
            "sampling": "Échantillonnage",
            "num_points": "Nombre de points à générer (ex : 1000)",
            "save_success": "Enregistrement réussi",
            "save_points": "Les points des gaussiennes ont été enregistrés dans :",
            "export_options": "Options d'export",
            "include_model_info": "Inclure les informations du modèle",

            # === Divers ===
            "yes": "Oui",
            "no": "Non",
        },

        "en": {
            # === Application ===
            "app_title": "Pyk?n?Fit",

            # === UI principale ===
            "select_sample": "Select sample:",
            "sample_title": "Sample title:",
            "show_data": "Show data",
            "calculate_model": "Compute model",

            # === Menus ===
            "file": "File",
            "process": "Process",
            "language": "Language",
            "quit": "Quit",

            "import_data": "Import data",
            "generate_example": "Generate sample data",

            "save_full_graph": "Save full graph",
            "save_decomposition_results": "Save decomposition results",
            "save_gaussian_points": "Save Gaussian points (Excel)",

            # === Context menu ===
            "edit": "Edit",
            "delete": "Delete",
            "clear": "Clear",
            "cut": "Cut",
            "copy": "Copy",
            "paste": "Paste",

            # === Graphes ===
            "left_click_help": "left click = add",
            "right_click_help": "right click = edit / delete",
            "x_label_phi": "x (?)",
            "y_label_frequency": "Frequency",
            "particle_diameter": "Particle diameter in µm (log)",
            "data": "Data",
            "best_fit": "Best fit",
            "absolute_error": "Absolute error",
            "relative_error": "Relative error",
            "MSE": "MSE",
            "R_square": "R²",

            # === Modélisation ===
```

```
            "multi_modal_title": "Multimodal decomposition of sample",

            # === Boîtes de dialogue ===
            "error": "Error",
            "info": "Information",
            "warning": "Warning",

            "error_graph": "An error occurred while generating the graph: ",
            "invalid_data": "Invalid input data.",
            "need_data": "You must enter data first.",
            "need_one_center": "You need at least one center.",
            "model_success": "Probabilistic decomposition completed",

            "no_file_selected": "No file selected.",
            "no_results": "No stored results yet.",
            "no_model": "No computed model.",
            "graph_saved": "Graph saved as",
            "results_saved": "Results saved as",

            # === Import / Export ===
            "choose_file": "Choose an Excel or CSV file",
            "unsupported_format": "Unsupported format",
            "sampling": "Sampling",
            "num_points": "Number of points to generate (e.g. 1000)",
            "save_success": "Save successful",
            "save_points": "The Gaussian points were recorded in:",
            "export_options": "Export options",
            "include_model_info": "Include model information",

            # === Divers ===
            "yes": "Yes",
            "no": "No",
        }
    }

```

Fichier: install.py

Chemin: .\install.py

```
import subprocess
import sys

# Liste des librairies nécessaires
packages = [
    "numpy",
    "pandas",
    "matplotlib",
    "scipy",
    "lmfit",
    "pillow",
    "openpyxl",   # pour l'import/export Excel
]

def install(package):
    subprocess.check_call([sys.executable, "-m", "pip", "install", package])

if __name__ == "__main__":
    print("Installation des dépendances...")
    for pkg in packages:
        try:
            print(f"? Installation de {pkg}...")
            install(pkg)
        except Exception as e:
            print(f"?? Erreur lors de l'installation de {pkg} : {e}")

    print("\n?? Installation terminée.")

```

Fichier: main.py

Chemin: .\main.py

```
import tkinter as tk
from app import PyknFitApp
import traceback

if __name__ == "__main__":
    try:
        root = tk.Tk()
        app = PyknFitApp(root)
        root.mainloop()

    except Exception as e:
        print("Une erreur est survenue :")
        traceback.print_exc()

    input("\nAppuie sur Entrée pour fermer...")
```

Fichier: plotting.py

Chemin: .\plotting.py

```
import tkinter as tk
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tkinter import Menu, simpledialog, Toplevel, ttk, Tk, messagebox
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
from scipy.interpolate import interp1d
import re


class PlotManager:
    def __init__(self, app):
        self.app = app

    def extract_sigma_value(self, text):
        m = re.search(r'?\s*=\s*([\d.]+)', text)
        return float(m.group(1)) if m else None

    def extract_amplitude_value(self, text):
        for line in text.split("\n"):
            if line.startswith("A="):
                try:
                    return float(line.split("=")[1])
                except:
                    return None
        return None

    def on_click(self, event):
        if event.inaxes:
            if event.button == 1 and not self.app.m_edit_point_menu_open:
                x_new = event.xdata
                y_new = self.app.interpolator(x_new)
                self.app.m_dynamic_x.append(x_new)
                self.app.m_dynamic_y.append(y_new)
                self.app.m_init_centers_selected.append(round(x_new, 1))

                annotation = self.app.m_ax0.annotate(
                    f"?=1.0\nA={np.round(y_new,1)}",
                    (x_new, y_new),
                    textcoords="offset points", xytext=(5, -15),
                    ha='left', fontsize=10, color='red'
                )
                self.app.m_annotations[x_new] = annotation

            elif event.button == 3:
                if self.app.m_dynamic_x:
                    distances = [(x - event.xdata) ** 2 + (y - event.ydata) ** 2
                                 for x, y in zip(self.app.m_dynamic_x, self.app.m_dynamic_y)]
                    nearest_index = distances.index(min(distances))
                    x_selected_point = self.app.m_dynamic_x[nearest_index]
                    annotation = self.app.m_annotations[x_selected_point].get_text()
                    self.show_context_menu(event, nearest_index, x_selected_point, annotation)

            if self.app.m_dynamic_x:
                self.app.scatter.set_offsets(list(zip(self.app.m_dynamic_x, self.app.m_dynamic_y)))
            else:
                self.app.scatter.set_offsets(np.empty((0, 2)))
            event.canvas.draw_idle()

    def show_context_menu(self, event, index, x_selected_point, annotation):
```

```
            self.app.m_edit_point_menu_open = True
            root = Tk()
            root.withdraw()
            menu = Menu(root, tearoff=0)
            menu.add_command(label=self.app.t("edit"), command=lambda: self.edit_point(index, ...
            menu.add_command(label=self.app.t("delete"), command=lambda: self.delete_point(index...
            menu.post(event.guiEvent.x_root, event.guiEvent.y_root)
            root.mainloop()

        def edit_point(self, index, x_selected_point, annotation):
            root = Tk()
            root.withdraw()
            temp_window = Toplevel(root)
            temp_window.withdraw()
            temp_window.grab_set()
            temp_window.lift()
            sigma = simpledialog.askfloat(self.app.t("edit") + " ?", "?",
                                          initialvalue=self.extract_sigma_value(annotation),
                                          parent=temp_window)
            amplitude = simpledialog.askfloat(self.app.t("edit") + " A", "A",
                                              initialvalue=self.extract_amplitude_value(annotation),
                                              parent=temp_window)
            temp_window.destroy()
            if sigma is not None and amplitude is not None:
                self.app.m_annotations[x_selected_point].set_text(f"?={sigma:.1f}\nA={amplitude:.1f}")
                plt.draw()
            self.app.m_edit_point_menu_open = False

        def delete_point(self, index, x_selected_point):
            self.app.m_dynamic_x.pop(index)
            self.app.m_dynamic_y.pop(index)
            self.app.m_init_centers_selected.pop(index)
            if x_selected_point in self.app.m_annotations:
                self.app.m_annotations[x_selected_point].remove()
                del self.app.m_annotations[x_selected_point]
            if self.app.m_dynamic_x:
                self.app.scatter.set_offsets(list(zip(self.app.m_dynamic_x, self.app.m_dynamic_y)))
            else:
                self.app.scatter.set_offsets(np.empty((0, 2)))
            plt.draw()
            self.app.m_edit_point_menu_open = False

        def afficher_graphique_donnees_wrapper(self):
            if (self.app.m_titre_entry.get()
                and self.app.m_textX.get("1.0", "end").strip()
                and self.app.m_textY.get("1.0", "end").strip()):
                self.afficher_graphique_donnees()
                self.app.m_process_menu.entryconfig(self.app.m_entry_save_graph,   state="normal")
                self.app.m_process_menu.entryconfig(self.app.m_entry_save_results, state="disabled")
                self.app.m_process_menu.entryconfig(self.app.m_entry_save_gauss,   state="disabled")
            else:
                messagebox.showerror(self.app.t("error"), self.app.t("need_data"))

        def afficher_graphique_donnees(self):
            self.app.model.calcul_resultats()

            for w in self.app.ui.graph_frame.winfo_children():
                w.destroy()

            self.app.m_fig0, self.app.m_ax0 = plt.subplots(figsize=(6, 4), dpi=150)
            self.app.m_ax0.plot(self.app.m_chiffresX_phi, self.app.m_chiffresY_μ, '-')
            self.app.m_ax0.set_title(self.app.m_titre_graphique + "\n(" + self.app.t("left_click_hel...
            self.app.m_ax0.set_xlabel(self.app.t("x_label_phi"))
            self.app.m_ax0.set_ylabel(self.app.t("y_label_frequency"))
            self.app.m_ax0.invert_xaxis()
```

```
            self.app.m_ax0.grid(True)

            self.app.scatter = self.app.m_ax0.scatter(self.app.m_dynamic_x, self.app.m_dynamic_y, co...

            self.app.interpolator = interp1d(self.app.m_chiffresX_phi, self.app.m_chiffresY_µ,
                                             kind='linear', bounds_error=False, fill_value="extrapolate")

            self.app.m_fig0.canvas.mpl_connect("button_press_event", self.on_click)

            canvas = FigureCanvasTkAgg(self.app.m_fig0, master=self.app.ui.graph_frame)
            canvas.draw()
            canvas.get_tk_widget().pack()

            if not self.app.bouton_calculer_visible:
                frame_boutons = tk.Frame(self.app.root)
                frame_boutons.grid(row=5, column=0, padx=10, pady=10)
                self.app.root.grid_columnconfigure(0, weight=1)
                frame_boutons.grid_configure(sticky='n')

                label_dist = tk.Label(frame_boutons, text="Distribution :")
                label_dist.pack(side=tk.LEFT, padx=(0, 5))
                self.app.distribution_type = ttk.Combobox(frame_boutons, values=['Gaussienne', 'Log-n...
                self.app.distribution_type.current(0)
                self.app.distribution_type.pack(side=tk.LEFT, padx=(0, 10))

                self.app.bouton_calculer = tk.Button(
                    frame_boutons, text=self.app.t("calculate_model"),
                    command=self.afficher_graphique_resultats_wrapper
                )
                self.app.bouton_calculer.pack(side=tk.LEFT)

                self.app.bouton_calculer_visible = True

        def afficher_graphique_resultats_wrapper(self, mode=1):
            if len(self.app.m_init_centers_selected) >= 1:
                self.app.model.calcul_resultats()
                self.afficher_graphique_resultats()
                self.app.m_process_menu.entryconfig(self.app.m_entry_save_graph,   state="normal")
                self.app.m_process_menu.entryconfig(self.app.m_entry_save_results, state="normal")
                self.app.m_process_menu.entryconfig(self.app.m_entry_save_gauss,   state="normal")
            else:
                messagebox.showerror(self.app.t("error"), self.app.t("need_one_center"))

        def ajouter_valeurs_centrales(self, ax):
            composantes = []

            for i, p in enumerate(self.app.m_model_params, start=1):
                # Récupérer la distribution correspondante
                dist = self.app.model.distributions[i-1] if hasattr(self.app.model, 'distributions')...

                # Déterminer les paramètres selon le type de distribution
                info = dist.table_info(p)

                composantes.append({
                    "index": i,
                    "phi": info["phi"],
                    "amplitude": p["amplitude"],
                    "hauteur": info["hauteur"],
                    "micron": info["micron"],
                    "params_table": info["params_table"]
                })

            # Trier par diamètre (ordre croissant)
            composantes = sorted(composantes, key=lambda c: c['micron'])

```

```
            # Mettre à jour les listes de l'app
            self.app.m_centres_microns = [c['micron'] for c in composantes]
            self.app.m_centres_phi = [c['phi'] for c in composantes]
            self.app.m_distribution_params = [
                c["params_table"]
                for c in composantes
            ]
            self.app.m_amplitudes = [c['hauteur'] for c in composantes]

            # Calculer les proportions
            self.app.m_proportions = self.app.model.calculer_aires_et_proportions()

            # Créer le DataFrame pour le tableau
            rows = []
            for c in composantes:
                row = {
                    '?m': round(c['micron'], 2),
                }
                row.update({
                    k: round(v, 2)
                    for k, v in c['params_table'].items()
                })
                row['H'] = round(c['hauteur'], 2)
                rows.append(row)

            df = pd.DataFrame(rows)
            # df['Diamètre (µm)'] = df['Diamètre (µm)'].astype(float)

            # Afficher la légende/tableau
            self.afficher_legende_parameters(ax, df)

            return [c['index'] for c in composantes]  # ordre des indices

        def afficher_canvas(self, fig, graph_frame):
            canvas = FigureCanvasTkAgg(fig, master=graph_frame)
            canvas.draw()
            canvas.get_tk_widget().pack()
            return canvas

        def afficher_legende_parameters(self, _ax, _param_df):
            table = _ax.table(cellText=_param_df.values, colLabels=_param_df.columns, loc='upper...
            table.auto_set_font_size(False)
            table.set_fontsize(8)

            for key, cell in table.get_celld().items():
                cell.set_linewidth(0)

            l_curves_default_colors = plt.rcParams['axes.prop_cycle'].by_key()['color']
            for i in range(len(_param_df)):
                color = l_curves_default_colors[(i+1) % len(l_curves_default_colors)]
                for j in range(len(_param_df.columns)):
                    table[i + 1, j].get_text().set_color(color)

        def afficher_graphique_resultats(self):
            try:
                for widget in self.app.ui.graph_frame.winfo_children():
                    widget.destroy()

                self.app.m_fig2, self.app.m_ax2 = plt.subplots(figsize=(6, 4), dpi=150)

                order_indices = self.ajouter_valeurs_centrales(self.app.m_ax2)
                self.app.m_proportions = (
                    self.app.m_model.calculer_aires_et_proportions()
                )
```

```
                distribution_type = getattr(
                    self.app,
                    'current_distribution_type',
                    'Gaussienne'
                )

                use_log_scale = (
                    len(self.app.m_model.distributions) > 0
                    and self.app.m_model.distributions[0].use_log_scale
                )

                if use_log_scale:
                    x_dense = np.geomspace(
                        max(min(self.app.m_fig1_X), 1e-6),
                        max(self.app.m_fig1_X),
                        self.app.m_sampling_model_size
                    )
                else:
                    x_dense = np.linspace(
                        min(self.app.m_fig1_X),
                        max(self.app.m_fig1_X),
                        self.app.m_sampling_model_size
                    )

                self.app.m_ax2.plot(self.app.m_fig1_X, self.app.m_fig1_Y, 'o',
                                    color='#99002299', markersize=4,
                                    label=self.app.t("data"), zorder=19)
                self.m_best_fit_curve = self.app.m_ax2.plot(
                    x_dense, self.app.m_model.eval(x=x_dense), '-',
                    label=self.app.t("best_fit"), zorder=20
                )

                if use_log_scale:
                    self.app.m_ax2.set_xscale("log")
                    self.app.m_ax2.set_xlabel("Diamètre (µm)")
                    self.app.m_ax2.set_xlabel(self.app.t("particle_diameter"))
                    self.app.m_ax2.set_ylabel(self.app.t("y_label_frequency"), fontsize=8)
                else:
                    self.app.m_ax2.invert_xaxis()
                    self.app.m_ax2.set_xlim([min(self.app.m_fig1_X), max(self.app.m_fig1_X)])
                    self.app.m_ax2.set_xticks(self.app.m_ticks)
                    self.app.m_ax2.set_xticklabels(self.app.m_ticklabels)
                    self.app.m_ax2.set_xlabel(self.app.t("particle_diameter"))
                    self.app.m_ax2.set_ylabel(self.app.t("y_label_frequency"), fontsize=8)
                    self.app.m_ax2.invert_xaxis()

                distribution_type = getattr(
                    self.app,
                    'current_distribution_type',
                    'Gaussienne'
                )

                components_eval = self.app.m_model.eval_components(x_dense)
                for rank, i in enumerate(order_indices):
                    name = f'g{i}_'
                    y_component = components_eval[name]
                    self.app.m_ax2.plot(x_dense, y_component,
                                        label=f'Component {rank+1}', linewidth=2)

                    idx_max = np.argmax(y_component)
                    x_peak = x_dense[idx_max]
                    y_peak = y_component[idx_max]
                    self.app.m_ax2.text(x_peak, y_peak * 0.5,
                                        f'{self.app.m_proportions[i-1]*100:.0f}%',
                                        ha='center', va='center', fontsize=8)
```

```
321
322                 l_error_texte = self.afficher_model_legend()
323
324                 # Titre principal au niveau de la figure (en haut)
325                 self.app.m_fig2.suptitle(
326                     self.app.t("multi_modal_title") + f" {self.app.m_titre_graphique}",
327                     fontsize=10,
328                     y=0.95  # Ajustez la position verticale
329                 )
330
331                 # Sous-titre (erreur) juste en dessous du titre principal
332                 self.app.m_ax2.set_title(l_error_texte, fontsize=7, pad=2)
333
334                 # Bloc d'informations du modèle (conditionnel)
335                 if getattr(self.app, "m_include_model_info", True):
336                     texte = (
337                         self.app.m_model.formula_description()
338                         + '\n\n'
339                         + self.app.m_model.derived_quantities_description()
340                     )
341                     self.app.m_fig2.text(
342                         0.5, 0.02, texte,
343                         ha='center', va='bottom',
344                         multialignment='left',
345                         fontsize=6, family='monospace',
346                         bbox=dict(facecolor='white', edgecolor='black', alpha=0.9)
347                     )
348                     self.app.m_fig2.subplots_adjust(bottom=0.38)
349                 else:
350                     self.app.m_fig2.subplots_adjust(bottom=0.15)
351
352                 canvas2 = self.afficher_canvas(self.app.m_fig2, self.app.ui.graph_frame)
353                 self.app.ui.graph_frame.update_idletasks()
354
355             except Exception as e:
356                 import traceback
357                 traceback.print_exc()
358                 messagebox.showinfo(
359                     self.app.t("error"),
360                     self.app.t("error_graph") + f"\n{e}"
361                 )
362
363         def afficher_model_legend(self):
364             handles, labels = self.app.m_ax2.get_legend_handles_labels()
365             filtered_handles = []
366             filtered_labels = []
367
368             for h, l in zip(handles, labels):
369                 if l in [self.app.t("data"), self.app.t("best_fit")]:
370                     filtered_handles.append(h)
371                     filtered_labels.append(l)
372
373             self.app.m_ax2.legend(filtered_handles, filtered_labels, loc='best', fontsize=6)
374
375             distribution_type = getattr(self.app, 'current_distribution_type', 'Gaussienne')
376
377             l_error_texte = (
378                 "("+f'Distribution : {distribution_type} / ' +
379                 self.app.t("absolute_error")+f"={self.app.m_last_error_metrics['Erreur absolue']:...
380                 self.app.t("relative_error")+f"={self.app.m_last_error_metrics['Erreur relativ...
381                 self.app.t("MSE")+f"={self.app.m_last_error_metrics['MSE']:.4f} / " +
382                 self.app.t("R_square")+f"={self.app.m_last_error_metrics['R²']:.4f})"
383             )
384
385             return l_error_texte
```

Fichier: test_engine.py

Chemin: .\test_engine.py

```
import numpy as np
import matplotlib.pyplot as plt

from pyknfit.distributions.gaussian import Gaussian
from pyknfit.distributions.weibull import Weibull

from pyknfit.mixtures.mixture_model import MixtureModel

x = np.linspace(0.01, 10, 500)

y = (
    0.6 * Gaussian().pdf(
        x,
        {"mu": 3, "sigma": 0.5}
    )
    +
    0.4 * Weibull().pdf(
        x,
        {"k": 2, "lambda": 6}
    )
)

model = MixtureModel([
    Gaussian(),
    Weibull()
])

model.fit(x, y)

y_pred = model.evaluate(x)

plt.plot(x, y)
plt.plot(x, y_pred)

plt.show()
```

Fichier: ui.py

Chemin: .\ui.py

```
from PIL import Image, ImageTk
import tkinter as tk
from tkinter import ttk, Menu, Label, messagebox

class UIManager:
    def __init__(self, app):
        self.app = app

    def create_widgets(self):
        self.frame_titre = tk.Frame(self.app.root)

        logo_img = Image.open(self.app.logo_path)
        logo_img = logo_img.resize((80, 80))
        self.logo = ImageTk.PhotoImage(logo_img)

        logo_label = tk.Label(self.frame_titre, image=self.logo)
        logo_label.grid(row=0, column=2, rowspan=2, padx=10)

        self.frame_titre.grid(row=0, column=0, padx=10, pady=10)

        tk.Label(self.frame_titre, text=self.app.t("select_sample")).grid(row=0, column=0, s...
        self.app.selection_echantillon = ttk.Combobox(self.frame_titre, values=[], state="...
        self.app.selection_echantillon.grid(row=0, column=1, padx=5, pady=2)

        tk.Label(self.frame_titre, text=self.app.t("sample_title")).grid(row=1, column=0, sticky="e")
        self.app.m_titre_entry = tk.Entry(self.frame_titre, width=40)
        self.app.m_titre_entry.grid(row=1, column=1, padx=5, pady=2)

        self.app.frameX = self.create_list_frame("x :", 1, self.app.root)
        self.app.m_textX = self.create_text_widget(self.app.frameX, 10, 40)
        self.app.uniteX = ttk.Combobox(self.app.frameX, values=["µ", "?"], state="readonly", width=3)
        self.app.uniteX.current(0)
        self.app.uniteX.grid(row=0, column=3)

        self.app.frameY = self.create_list_frame("y :", 2, self.app.root)
        self.app.m_textY = self.create_text_widget(self.app.frameY, 10, 40)
        Label(self.app.frameY, text="", width=5).grid(row=0, column=4)

        self.graph_frame = tk.Frame(self.app.root)
        self.graph_frame.grid(row=0, column=1, rowspan=6, padx=10, pady=10)

        self.app.bouton_afficher_donnees = tk.Button(
            self.app.root, text=self.app.t("show_data"),
            command=self.app.plot.afficher_graphique_donnees_wrapper
        )
        self.app.bouton_afficher_donnees.grid(row=4, column=0, padx=10, pady=10)

        self.app.progress = ttk.Progressbar(self.app.root, orient="horizontal", length=300,...
        self.app.progress.grid(row=6, column=0, padx=10, pady=10)

        self.app.menu_contextuel = tk.Menu(self.app.root, tearoff=0)

    def create_menu(self):
        menuBar = tk.Menu(self.app.root)

        # ============================================================
        # Menu Fichier
        # ============================================================
        file_menu = tk.Menu(menuBar, tearoff=0)
        file_menu.add_command(
```

```
                label=self.app.t("import_data"),
                command=self.app.data.load_data
            )
            file_menu.add_command(
                label=self.app.t("generate_example"),
                command=self.app.data.insert_input_data
            )
            file_menu.add_separator()
            file_menu.add_command(
                label=self.app.t("quit"),
                command=self.app.root.destroy
            )

            # ============================================================
            # Menu Traitement
            # ============================================================
            process_menu = tk.Menu(menuBar, tearoff=0)
            process_menu.add_command(
                label=self.app.t("show_data"),
                command=self.app.plot.afficher_graphique_donnees_wrapper
            )
            process_menu.add_separator()

            # --- Entrées d'export (désactivées au départ) ---------------
            # ?? add_command() renvoie None : on récupère l'index via menu.index("end")
            process_menu.add_command(
                label=self.app.t("save_full_graph"),
                command=self.app.data.enregistrer_graphique,
                state="disabled"
            )
            self.app.m_entry_save_graph = process_menu.index("end")

            process_menu.add_command(
                label=self.app.t("save_decomposition_results"),
                command=self.app.data.enregistrer_resultats_decomposition,
                state="disabled"
            )
            self.app.m_entry_save_results = process_menu.index("end")

            process_menu.add_command(
                label=self.app.t("save_gaussian_points"),
                command=self.app.data.enregistrer_points_gaussiennes_excel,
                state="disabled"
            )
            self.app.m_entry_save_gauss = process_menu.index("end")

            # --- Options d'export : case à cocher -----------------------
            process_menu.add_separator()
            self.app.var_include_model_info = tk.BooleanVar(
                value=getattr(self.app, "m_include_model_info", True)
            )
            process_menu.add_checkbutton(
                label=self.app.t("include_model_info"),
                variable=self.app.var_include_model_info,
                command=self.toggle_include_model_info
            )

            # ============================================================
            # Menu Langue
            # ============================================================
            lang_menu = tk.Menu(menuBar, tearoff=0)
            lang_menu.add_command(
                label="Français",
                command=lambda: self.app.set_language("fr")
            )
```

```
            lang_menu.add_command(
                label="English",
                command=lambda: self.app.set_language("en")
            )

            # ============================================================
            # Assemblage
            # ============================================================
            menuBar.add_cascade(label=self.app.t("file"),     menu=file_menu)
            menuBar.add_cascade(label=self.app.t("process"),  menu=process_menu)
            menuBar.add_cascade(label=self.app.t("language"), menu=lang_menu)

            self.app.root.config(menu=menuBar)

            # Références conservées pour activation/désactivation ultérieure
            self.app.m_file_menu    = file_menu
            self.app.m_process_menu = process_menu

        def create_context_menu(self):
            for action in [self.app.t("clear"), self.app.t("cut"), self.app.t("copy"), self.app.t(...
                self.app.menu_contextuel.add_command(label=action)

            self.bind_context_menu(self.app.m_textX)
            self.bind_context_menu(self.app.m_textY)

        def bind_context_menu(self, widget):
            widget.bind("<Button-3>", lambda event: self.afficher_menu_contextuel(event, widget))

        def afficher_menu_contextuel(self, event, text_widget):
            self.app.menu_contextuel.entryconfig(self.app.t("clear"), command=lambda: text_widget.de...
            self.app.menu_contextuel.entryconfig(self.app.t("cut"), command=lambda: text_widget.event_...
            self.app.menu_contextuel.entryconfig(self.app.t("copy"), command=lambda: text_widg...
            self.app.menu_contextuel.entryconfig(self.app.t("paste"), command=lambda: text_widge...
            self.app.menu_contextuel.tk_popup(event.x_root, event.y_root)

        def create_list_frame(self, label_text, row, root):
            frame = tk.Frame(root)
            frame.grid(row=row, column=0, padx=10, pady=10)
            tk.Label(frame, text=label_text).grid(row=0, column=0)
            return frame

        def create_text_widget(self, frame, height, width):
            text_widget = tk.Text(frame, height=height, width=width)
            text_widget.grid(row=0, column=1)
            scrollbar = tk.Scrollbar(frame, command=text_widget.yview)
            scrollbar.grid(row=0, column=2, sticky="ns")
            text_widget.config(yscrollcommand=scrollbar.set)
            return text_widget

        def toggle_include_model_info(self):
            """
            Appelée quand l'utilisateur coche/décoche
            'Inclure les informations du modèle'.

            - Met à jour l'état persistant dans l'app.
            - Redessine le graphique des résultats si un modèle existe déjà,
              pour que l'encadré apparaisse/disparaisse immédiatement.
            """
            # 1) Synchroniser l'état persistant avec la BooleanVar
            self.app.m_include_model_info = self.app.var_include_model_info.get()

            # 2) Rafraîchir le graphique des résultats si nécessaire
            if self.app.m_fig2 is not None and self.app.m_model is not None:
                try:
                    self.app.plot.afficher_graphique_resultats()
```

```
                except Exception as e:
                    import traceback
                    traceback.print_exc()
                    messagebox.showerror(
                        self.app.t("error"),
                        self.app.t("error_graph") + f"\n{e}"
                    )
```

Fichier: __init__.py

Chemin: .\pyknfit__init__.py

```
1       """PyknFit - Probabilistic multimodal decomposition framework"""
```

Fichier: metrics.py

Chemin: .\pyknfit\diagnostics\metrics.py

```
import numpy as np

def mse(y_true, y_pred):

    return np.mean((y_true - y_pred) ** 2)


def r_squared(y_true, y_pred):

    ss_res = np.sum((y_true - y_pred) ** 2)

    ss_tot = np.sum((y_true - np.mean(y_true)) ** 2)

    return 1 - (ss_res / ss_tot)


def aic(loglikelihood, k):

    return 2 * k - 2 * loglikelihood


def bic(loglikelihood, k, n):

    return k * np.log(n) - 2 * loglikelihood

```

Fichier: __init__.py

Chemin: .\pyknfit\diagnostics__init__.py

```
1
```

Fichier: base.py

Chemin: .\pyknfit\distributions\base.py

```
from abc import ABC, abstractmethod
import numpy as np

class Distribution(ABC):

    @property
    def use_log_scale(self):
        return False

    @abstractmethod
    def pdf(self, x, params):
        pass

    @abstractmethod
    def logpdf(self, x, params):
        pass

    @abstractmethod
    def sample(self, n, params):
        pass

    @abstractmethod
    def initialize(self, x, y):
        pass

    @abstractmethod
    def create_initial_params(
        self,
        center,
        sigma,
        amplitude
    ):
        pass

    @abstractmethod
    def pack_theta(self, params):
        pass

    @abstractmethod
    def unpack_theta(self, theta):
        pass

    @abstractmethod
    def bounds(self):
        pass

    @abstractmethod
    def gui_center_value(self, params):
        pass

    @abstractmethod
    def gui_width_value(self, params):
        pass

    @abstractmethod
    def table_info(self, params):
        pass

    @abstractmethod
    def formula_description(self):
```

```
            pass

        @abstractmethod
        def derived_quantities_description(self):
            pass
```

Fichier: gaussian.py

Chemin: .\pyknfit\distributions\gaussian.py

```
import numpy as np
from scipy.stats import norm
from .base import Distribution

class Gaussian(Distribution):

    def pdf(self, x, params):
        mu = params["mu"]
        sigma = params["sigma"]

        return norm.pdf(x, loc=mu, scale=sigma)

    def logpdf(self, x, params):
        mu = params["mu"]
        sigma = params["sigma"]

        return norm.logpdf(x, loc=mu, scale=sigma)

    def sample(self, n, params):
        mu = params["mu"]
        sigma = params["sigma"]

        return np.random.normal(mu, sigma, n)

    def initialize(self, x, y):

        idx = np.argmax(y)

        return {
            "mu": x[idx],
            "sigma": np.std(x)
        }

    def create_initial_params(
        self,
        center,
        sigma,
        amplitude
    ):
        return {
            "mu": center,
            "sigma": sigma,
            "amplitude": amplitude
        }

    def pack_theta(self, params):
        return [
            params["mu"],
            params["sigma"],
            params["amplitude"]
        ]

    def unpack_theta(self, theta):
        return {
            "mu": theta[0],
            "sigma": abs(theta[1]),
            "amplitude": abs(theta[2])
        }

    def bounds(self):
```

```
            return [
                (None, None),
                (0.05, None),
                (0.0, None)
            ]

        def gui_center_value(self, params):
            return params["mu"]

        def gui_width_value(self, params):
            return params["sigma"]

        def table_info(self, params):

            mu = params["mu"]
            sigma = params["sigma"]
            amp = params["amplitude"]

            micron = 1000 * (2 ** (-mu))

            hauteur = amp / (
                sigma * np.sqrt(2*np.pi)
            )

            return {
                "micron": micron,
                "phi": mu,
                "hauteur": hauteur,
                "params_table": {
                    "?": sigma
                }
            }

        def formula_description(self):
            return (
                "f(x)=H/(??(2?)) exp(-(x-?)²/(2?²))\n\n"
                "Paramètres du modèle :\n"
                "? : moyenne (centre)\n"
                "? : écart-type\n"
                "H : aire (poids) de la composante"
            )

        def derived_quantities_description(self):
            return (
                "Grandeurs dérivées :\n"
                "µm = 1000 × 2^(-?)"
            )
```

Fichier: lognormal.py

Chemin: .\pyknfit\distributions\lognormal.py

```
import numpy as np
from scipy.stats import lognorm
from .base import Distribution

class LogNormal(Distribution):

    @property
    def use_log_scale(self):
        return True

    def pdf(self, x, params):

        sigma = params["sigma"]
        scale = np.exp(params["mu"])

        return lognorm.pdf(x, s=sigma, scale=scale)

    def logpdf(self, x, params):

        sigma = params["sigma"]
        scale = np.exp(params["mu"])

        return lognorm.logpdf(x, s=sigma, scale=scale)

    def sample(self, n, params):

        mu = params["mu"]
        sigma = params["sigma"]

        return np.random.lognormal(mu, sigma, n)

    def initialize(self, x, y):

        positive_x = x[x > 0]

        return {
            "mu": np.mean(np.log(positive_x)),
            "sigma": np.std(np.log(positive_x))
        }

    def create_initial_params(
        self,
        center,
        sigma,
        amplitude
    ):
        return {
            "mu": np.log(center),
            "sigma": sigma,
            "amplitude": amplitude
        }

    def pack_theta(self, params):
        return [
            params["mu"],
            params["sigma"],
            params["amplitude"]
        ]

    def unpack_theta(self, theta):
```

```
            return {
                "mu": theta[0],
                "sigma": abs(theta[1]),
                "amplitude": abs(theta[2])
            }

        def bounds(self):
            return [
                (None, None),   # mu
                (0.05, None),   # sigma
                (0.0, None)     # amplitude
            ]

        def gui_center_value(self, params):
            return params["mu"]

        def gui_width_value(self, params):
            return params["sigma"]

        def table_info(self, params):
            mu = params["mu"]
            sigma = params["sigma"]
            amp = params["amplitude"]

            # diamètre modal
            micron = np.exp(mu - sigma**2)

            # hauteur maximale
            hauteur = (
                amp *
                self.pdf(
                    np.array([micron]),
                    params
                )[0]
            )

            return {
                "micron": micron,
                "phi": mu,
                "hauteur": hauteur,
                "params_table": {
                    "?": sigma
                }
            }

        def formula_description(self):
            return (
                "f(x)=H/(x??(2?)) exp(-(ln(x)-?)²/(2?²))\n\n"
                "Paramètres du modèle :\n"
                "? : moyenne de ln(x)\n"
                "? : écart-type de ln(x)\n"
                "H : aire (poids) de la composante"
            )

        def derived_quantities_description(self):
            return (
                "Grandeurs dérivées :\n"
                "µm = exp(?)"
            )
```

Fichier: weibull.py

Chemin: .\pyknfit\distributions\weibull.py

```
import numpy as np
from scipy.stats import weibull_min
from .base import Distribution

class Weibull(Distribution):

    @property
    def use_log_scale(self):
        return True

    def pdf(self, x, params):
        k = params["k"]
        lam = params["lambda"]
        return weibull_min.pdf(x, c=k, scale=lam)

    def logpdf(self, x, params):
        k = params["k"]
        lam = params["lambda"]
        return weibull_min.logpdf(x, c=k, scale=lam)

    def sample(self, n, params):
        k = params["k"]
        lam = params["lambda"]
        return weibull_min.rvs(c=k, scale=lam, size=n)

    def mode(self, params):
        k = params["k"]
        lam = params["lambda"]

        if k <= 1:
            return np.nan

        return lam * ((k - 1) / k) ** (1.0 / k)

    def mean(self, params):
        from scipy.special import gamma

        k = params["k"]
        lam = params["lambda"]

        return lam * gamma(1 + 1/k)

    def initialize(self, x, y):
        idx = np.argmax(y)
        return {
            "k": 2.0,
            "lambda": x[idx]
        }

    def create_initial_params(
        self,
        center,
        sigma,
        amplitude
    ):
        return {
            "k": 2.0,
            "lambda": center,
            "amplitude": amplitude
        }
```

```
61
62          def pack_theta(self, params):
63              return [
64                  params["k"],
65                  params["lambda"],
66                  params["amplitude"]
67              ]
68
69          def unpack_theta(self, theta):
70              return {
71                  "k": abs(theta[0]),
72                  "lambda": abs(theta[1]),
73                  "amplitude": abs(theta[2])
74              }
75
76          def bounds(self):
77              return [
78                  (1.01, 8.0),
79                  (0.01, None),
80                  (0.0, None)
81              ]
82
83          def gui_center_value(self, params):
84              return params["k"]
85
86          def gui_width_value(self, params):
87              return params["lambda"]
88
89          def table_info(self, params):
90
91              amp = params["amplitude"]
92
93              micron = self.mode(params)
94
95              hauteur = (
96                  amp *
97                  self.pdf(
98                      np.array([micron]),
99                      params
100                 )[0]
101             )
102
103             return {
104                 "micron": micron,
105                 "phi": np.nan,
106                 "hauteur": hauteur,
107                 "params_table": {
108                     "?": params["lambda"],
109                     "k": params["k"]
110                 }
111             }
112
113         def formula_description(self):
114             return (
115                 "f(x)=H(k/?)(x/?)^(k-1) exp(-(x/?)^k)\n\n"
116                 "Paramètres du modèle :\n"
117                 "? : paramètre d'échelle\n"
118                 "k : paramètre de forme\n"
119                 "H : aire (poids) de la composante"
120             )
121
122         def derived_quantities_description(self):
123             return (
124                 "Grandeurs dérivées :\n"
125                 "µm = ? ((k-1)/k)^(1/k)"
```

```
126                 )
```

Fichier: __init__.py

Chemin: .\pyknfit\distributions__init__.py

```
from .base import Distribution
from .gaussian import Gaussian
from .lognormal import LogNormal
from .weibull import Weibull

DISTRIBUTIONS = {
    "Gaussienne": Gaussian,
    "Log-normale": LogNormal,
    "Weibull": Weibull,
}

__all__ = [
    "Distribution",
    "Gaussian",
    "LogNormal",
    "Weibull",
    "DISTRIBUTIONS"
]
```

Fichier: mixture_model.py

Chemin: .\pyknfit\mixtures\mixture_model.py

```
import math
import numpy as np

from tkinter import messagebox
from scipy.optimize import minimize, least_squares
from pyknfit.distributions import DISTRIBUTIONS
from pyknfit.distributions.gaussian import Gaussian
from pyknfit.distributions.lognormal import LogNormal
from pyknfit.distributions.weibull import Weibull

class Parameter:

    def __init__(self, value):
        self.value = value


class MixtureModel:

    def __init__(self, app, distributions):

        self.app = app

        self.distributions = distributions

        self._params = []

        self.params_gui = {}

        self.components = []

        self.last_fit = None

    # ============================================================
    # Compatibilité GUI/lmfit
    # ============================================================

    @property
    def params(self):
        return self.params_gui

    # ============================================================
    # Lecture des données GUI
    # ============================================================

    def calcul_resultats(self):

        listeX = (
            self.app.m_textX
            .get("1.0", "end")
            .strip()
            .replace(",", ".")
        )

        listeY = (
            self.app.m_textY
            .get("1.0", "end")
            .strip()
            .replace(",", ".")
        )

```

```
            self.app.m_titre_graphique = (
                self.app.m_titre_entry.get()
            )

            try:

                self.app.m_chiffresX_µ = [
                    float(x)
                    for x in listeX.splitlines()
                ]

                self.app.m_chiffresY_µ = [
                    float(y)
                    for y in listeY.splitlines()
                ]

                # conversion µm -> phi
                if self.app.uniteX.get() == "µ":

                    self.app.m_chiffresX_phi = [

                        -math.log10(x / 1000)
                        / math.log10(2)

                        if x != 0 else 0

                        for x in self.app.m_chiffresX_µ
                    ]

                else:

                    self.app.m_chiffresX_phi = (
                        self.app.m_chiffresX_µ
                    )

                    self.app.m_chiffresY_phi = (
                        self.app.m_chiffresY_µ
                    )

            except ValueError:

                messagebox.showerror(
                    self.app.t("error"),
                    self.app.t("invalid_data")
                )

                return

            if len(self.app.m_init_centers_selected) >= 1:
                self.fit_gui_model()

        # ============================================================
        # Évaluation du modèle
        # ============================================================

        def evaluate(self, x):

            total = np.zeros_like(x, dtype=float)

            for dist, params in zip(
                self.distributions,
                self._params
            ):

                amp = params["amplitude"]
```

```
126
127                 total += (
128                     amp
129                     * dist.pdf(x, params)
130                 )
131
132             return total
133
134         def eval(self, x):
135
136             return self.evaluate(x)
137
138         # ============================================================
139         # Évaluation composantes
140         # ============================================================
141
142         def eval_components(self, x):
143
144             out = {}
145
146             for i, (dist, params) in enumerate(
147                 zip(
148                     self.distributions,
149                     self._params
150                 ),
151                 start=1
152             ):
153
154                 amp = params["amplitude"]
155
156                 out[f"g{i}_"] = amp * dist.pdf(x, params)
157
158             return out
159
160         # ============================================================
161         # Optimisation
162         # ============================================================
163         def fit(self, x, y):
164             n = len(self.app.m_init_centers_selected)
165
166             # Reconstruire les distributions selon le type choisi
167             dist_type = getattr(self.app, 'current_distribution_type', 'Gaussienne')
168             cls = DISTRIBUTIONS.get(dist_type, Gaussian)
169             self.distributions = [cls() for _ in range(n)]
170
171             self._params = []
172             keys = list(self.app.m_annotations.keys())
173
174             for i in range(n):
175                 center_val = self.app.m_init_centers_selected[i]
176
177                 from pyknfit.distributions.weibull import Weibull
178                 from pyknfit.distributions.lognormal import LogNormal
179
180                 if isinstance(self.distributions[i], (Weibull, LogNormal)):
181                     center_val = 1000 * (2 ** (-center_val))
182
183                 annotation = self.app.m_annotations[keys[i]].get_text()
184
185                 sigma = 1.0
186                 amplitude = 1.0
187
188                 for line in annotation.split("\n"):
189                     if line.startswith("?="):
190                         sigma = float(line.split("=")[1])
```

```
                        elif line.startswith("A="):
                            amplitude = float(line.split("=")[1])

                    sigma = max(sigma, self.app.m_min_sigma)

                    # Initialisation spécifique à la distribution
                    dist = self.distributions[i]
                    self._params.append(
                        dist.create_initial_params(
                            center=center_val,
                            sigma=sigma,
                            amplitude=amplitude
                        )
                    )

                # Préparation des paramètres initiaux pour l'optimisation
                initial = []
                for dist, p in zip(
                    self.distributions,
                    self._params
                ):
                    initial.extend(
                        dist.pack_theta(p)
                    )

                # Fonction de coût
                def objective(theta):
                    idx = 0
                    params = []

                    for dist in self.distributions:
                        n = len(dist.bounds())
                        theta_slice = theta[idx:idx+n]
                        params.append(
                            dist.unpack_theta(theta_slice)
                        )
                        idx += n

                    y_pred = np.zeros_like(x)
                    for dist, p in zip(self.distributions, params):
                        y_pred += p["amplitude"] * dist.pdf(x, p)

                    weights = 1.0 / (y + 0.05)
                    return np.mean(
                        weights * (y - y_pred) ** 2
                    )

                # Optimisation
                bounds = []
                for dist in self.distributions:
                    bounds.extend(
                        dist.bounds()
                    )

                result = minimize(
                    objective,
                    initial,
                    method='L-BFGS-B',
                    bounds=bounds
                )
                theta = result.x

                # Reconstruction des paramètres
                self._params = []
                idx = 0
```

```
            params = []

            for dist in self.distributions:

                n = len(dist.bounds())

                theta_slice = theta[idx:idx+n]

                params.append(
                    dist.unpack_theta(theta_slice)
                )

                idx += n

            self._params = params

            # Compatibilité GUI
            self.params_gui = {}
            for i, (dist, p) in enumerate(
                zip(self.distributions, self._params),
                start=1
            ):

                self.params_gui[f"g{i}_center"] = Parameter(
                    dist.gui_center_value(p)
                )

                self.params_gui[f"g{i}_sigma"] = Parameter(
                    dist.gui_width_value(p)
                )

                self.params_gui[f"g{i}_amplitude"] = Parameter(
                    p["amplitude"]
                )

        # ============================================================
        # Interface GUI
        # ============================================================

        def fit_gui_model(self):
            distribution_type = self.app.distribution_type.get()
            if distribution_type in ["Log-normale", "Weibull"]:
                x = np.array(self.app.m_chiffresX_μ)
            else:
                x = np.array(self.app.m_chiffresX_phi)

            y = np.array(self.app.m_chiffresY_?)  # ? changement ici : u ? ?
            n = len(self.app.m_init_centers_selected)

            # Récupérer la valeur du Combobox au moment du calcul
            self.app.current_distribution_type = self.app.distribution_type.get()
            distribution_type = self.app.current_distribution_type

            # Créer les distributions selon le type choisi
            dist_class = DISTRIBUTIONS[distribution_type]
            self.distributions = [
                dist_class()
                for _ in range(n)
            ]

            # Indicateur simple
            self.app.progress.config(mode='indeterminate')
            self.app.progress.start(10)
            self.app.root.update_idletasks()
```

```
            # Calcul
            self.fit(x, y)

            # Fin de l'indicateur
            self.app.progress.stop()
            self.app.progress.config(mode='determinate')
            self.app.progress["value"] = 100

            # Suite normale...
            self.app.m_fig1_X = x

            self.app.m_fig1_Y = y
            self.app.m_model = self
            self.app.m_model_params = self._params
            best_fit_y = self.eval(x)
            self.app.m_last_error_metrics = self.calculer_erreurs(y, best_fit_y, x)
            self.sauvegarder_resultats()

        # ============================================================
        # Proportions
        # ============================================================

        def calculer_aires_et_proportions(self):
            """Calcule les aires et proportions de chaque composante"""
            import numpy as np

            areas = []
            total = 0

            # Créer un axe x dense pour l'intégration numérique
            distribution_type = getattr(
                self.app,
                "current_distribution_type",
                "Gaussienne"
            )

            if hasattr(self.app, 'm_fig1_X') and self.app.m_fig1_X is not None:
                xmin = min(self.app.m_fig1_X)
                xmax = max(self.app.m_fig1_X)

                use_log_scale = (
                    len(self.app.m_model.distributions) > 0
                    and self.app.m_model.distributions[0].use_log_scale
                )

                if use_log_scale:
                    x_dense = np.geomspace(
                        max(xmin, 1e-6),
                        xmax,
                        5000
                    )
                else:
                    x_dense = np.linspace(
                        xmin,
                        xmax,
                        5000
                    )
            else:
                x_dense = np.linspace(-5, 5, 5000)

            for i, (dist, p) in enumerate(zip(self.distributions, self._params)):
                amp = p["amplitude"]

                # Calcul de l'aire par intégration numérique (plus robuste)
                y_comp = amp * dist.pdf(x_dense, p)
```

```
                area = np.trapezoid(y_comp, x_dense)

                areas.append(area)
                total += area

            if total <= 0:
                return [0] * len(self.distributions)

            return [a / total for a in areas]

        # ============================================================
        # Erreurs
        # ============================================================

        def calculer_erreurs(
            self,
            y_observe,
            y_modele,
            x
        ):

            erreur_absolue = np.trapezoid(
                np.abs(y_observe - y_modele),
                x
            )

            aire_observee = np.trapezoid(
                y_observe,
                x
            )

            erreur_relative = (
                100
                * erreur_absolue
                / aire_observee
                if aire_observee != 0
                else np.nan
            )

            mse = np.mean(
                (y_observe - y_modele) ** 2
            )

            ss_tot = np.sum(
                (y_observe - np.mean(y_observe)) ** 2
            )

            ss_res = np.sum(
                (y_observe - y_modele) ** 2
            )

            r_squared = (
                1 - (ss_res / ss_tot)
                if ss_tot != 0
                else np.nan
            )

            return {
                "Erreur absolue": erreur_absolue,
                "Erreur relative (%)": erreur_relative,
                "MSE": mse,
                "R²": r_squared
            }

        # ============================================================
```

```
        # Sauvegarde
        # ============================================================
        def sauvegarder_resultats(self):
            from pyknfit.distributions.lognormal import LogNormal
            from pyknfit.distributions.weibull import Weibull

            """Sauvegarde les résultats de la décomposition dans m_tous_les_resultats"""
            if self._params is None or len(self._params) == 0:
                return

            # Calculer les proportions
            proportions = self.calculer_aires_et_proportions()

            # Récupérer le type de distribution
            distribution_type = getattr(self.app, 'current_distribution_type', 'Gaussienne')

            # Récupérer les données
            centres_microns = []
            centres_phi = []
            sigmas = []
            hauteurs = []

            for i, p in enumerate(self._params, start=1):
                # from pyknfit.distributions.weibull import Weibull
                dist = self.distributions[i-1]

                if isinstance(dist, Weibull):
                    k = p['k']
                    sigma = p['lambda']

                    micron = sigma * ((k - 1)/k)**(1/k) if k > 1 else 0

                    phi = -np.log2(micron/1000) if micron > 0 else np.nan

                    if np.isfinite(micron) and micron > 0:
                        hauteur = p['amplitude'] * dist.pdf(micron, p)
                    else:
                        hauteur = 0
                elif isinstance(dist, LogNormal):
                    from pyknfit.distributions.lognormal import LogNormal

                    micron = np.exp(p["mu"])
                    phi = -np.log2(micron / 1000)
                    sigma = p["sigma"]
                    hauteur = p["amplitude"] * dist.pdf(micron, p)
                else:
                    phi = p['mu']
                    sigma = p['sigma']

                    micron = 1000 * (2 ** (-phi))

                    hauteur = p['amplitude'] / (sigma * np.sqrt(2*np.pi))

                centres_microns.append(micron)
                centres_phi.append(phi)
                sigmas.append(sigma)
                hauteurs.append(hauteur)

            # Trier par diamètre
            sorted_indices = sorted(range(len(centres_microns)), key=lambda i: centres_microns[i])

            # Supprimer les anciens résultats pour cet échantillon
            titre = self.app.m_titre_graphique
            self.app.m_tous_les_resultats = [
                r for r in self.app.m_tous_les_resultats
```

```
                if r.get("Echantillon") != titre
            ]

            # Ajouter les nouveaux résultats avec le type de distribution
            from datetime import datetime
            for idx in sorted_indices:
                self.app.m_tous_les_resultats.append({
                    "Echantillon": titre,
                    "Date": datetime.now().strftime("%d-%m-%Y %H:%M:%S"),
                    "Distribution": distribution_type,  # <-- AJOUTÉ
                    "Composante": idx + 1,
                    "Centre (µm)": centres_microns[idx],
                    "Proportion (%)": proportions[idx] * 100,  # En pourcentage
                    "Centre (phi)": centres_phi[idx],
                    "Amplitude": hauteurs[idx],
                    "Sigma": sigmas[idx],
                    **self.app.m_last_error_metrics
                })

        def formula_description(self):
            if len(self.distributions) == 0:
                return ""

            return self.distributions[0].formula_description()


        def derived_quantities_description(self):
            if len(self.distributions) == 0:
                return ""

            return self.distributions[0].derived_quantities_description()
```

Fichier: __init__.py

Chemin: .\pyknfit\mixtures__init__.py

```
from .mixture_model import MixtureModel

__all__ = ['MixtureModel']
```

Fichier: conversions.py

Chemin: .\pyknfit\utils\conversions.py

```
1
```

Fichier: __init__.py

Chemin: .\pyknfit\utils__init__.py

```
1
```